

ProjectOne Engineering Handbook

Chapter 4 — React Standards

4.1 Objective

React is used to build predictable, reusable and performant interfaces. Components must remain small, composable and easy to test.

4.2 Component Philosophy

Each component should have a single responsibility. Prefer composition over inheritance and split large components before they become difficult to understand.

4.3 Server vs Client Components

Default to Server Components where possible. Use Client Components only when browser APIs, local state or user interaction require them.

4.4 State Management

Keep state as local as possible. Avoid unnecessary global state. Derived state should be computed instead of duplicated.

4.5 Props

Props should be explicit, strongly typed and minimal. Avoid passing unrelated data or deeply nested prop chains.

4.6 Custom Hooks

Extract reusable logic into custom hooks. Hooks must encapsulate behavior rather than rendering and should expose a clean public API.

4.7 Performance

Avoid premature optimization. Use memoization only when profiling demonstrates measurable benefit. Prevent unnecessary re-renders through good component design.

4.8 Error Handling

Use Error Boundaries for unexpected UI failures. Loading, empty and error states must exist for asynchronous operations.

4.9 Accessibility

Every interactive element must be keyboard accessible, properly labeled and compatible with screen readers. Accessibility is a default requirement.

4.10 Styling

Use the shared design system and Tailwind conventions consistently. Avoid inline styles except for truly dynamic values.

4.11 Anti-Patterns

Avoid oversized components, duplicated logic, prop drilling when composition solves the problem, direct DOM manipulation and business logic inside presentation components.

4.12 Code Review Checklist

Verify readability, component size, hook usage, accessibility, performance implications, typing and adherence to the design system.

Chapter Summary

React code should remain modular, predictable and easy to evolve. Consistency across components is more valuable than individual coding preferences.